



Entrega Final

Compilador Claudio → Swift

Gramática ampliada · Reglas semánticas · SDT de traducción · Implementación · Pruebas

Convención visual	Uso en el informe
Rojo	Producciones de la gramática base.
Ámbar	Reglas semánticas: atributos, dominio y acción.
Azul	Acciones SDT que sintetizan lenguaje destino Swift.

Prueba directa del compilador

La aplicación desplegada está disponible en <https://claudio.dautia.com>. Los casos finales se pueden seleccionar desde la galería de programas y ejecutar en modo Semántico.

Autores: Juan David Cárdenas Jiménez · María José Restrepo Ramírez · José Manuel Zuluaga
Teoría de Compiladores · Facultad de Ingeniería · Semestre 1-2026

Tabla de Contenidos

Sección	Título	Pág.
1	Alcance del Compilador Final	3
2	Gramática Base y SDT	4
2.1–2.7	Producciones, semántica y acciones SDT	4–8
3	Reglas Semánticas Implementadas (SEM-1 ... SEM-7)	8
4	Cómo se Materializa en Claudio	9
5	Pruebas desde la Interfaz Gráfica	10
6	Bonus IA con OpenAI	13
7	Conclusiones	13

1 Alcance del Compilador Final

Claudio es un compilador fuente-a-fuente para un lenguaje en español que produce Swift como lenguaje destino. La entrega final integra análisis léxico, análisis sintáctico, análisis semántico y una SDT de traducción. La salida Swift se habilita únicamente cuando el programa supera las fases previas.

Requisito del enunciado	Evidencia en Claudio
Léxico → sintáctico → semántico → SDT	La compilación encadena fases y detiene la traducción cuando una fase queda inválida.
Errores unificados	Cada diagnóstico informa fase, ubicación, lexema asociado y mensaje comprensible.
Tabla de símbolos en semántica	El análisis conserva nombre, tipo, ámbito, mutabilidad, fila y columna para validar reglas SEM.
Casos de prueba desde UI	La galería incluye casos válidos primero y casos con fallas al final.
Swift como lenguaje destino	El panel Swift muestra contraste Claudio/Swift y estado de validación.
Bonus IA	OpenAI revisa la salida Swift generada sin reemplazar las reglas determinísticas del compilador.

Cadena de fases

Fase	Entrada	Proceso técnico	Salida
Léxica	Texto Claudio	Clasifica lexemas y conserva posición.	Tokens y errores léxicos.
Sintáctica	Tokens válidos	Aplica la gramática descendente recursiva y construye AST.	AST y diagnósticos sintácticos.
Semántica	AST	Valida tipos, ámbitos, constantes y condiciones.	Tabla de símbolos y reglas SEM.
SDT Swift	Programa válido	Ejecuta acciones de traducción asociadas a producciones.	Código Swift destino.
IA opcional	Swift generado	Revisa legibilidad y coherencia del destino.	Resumen complementario.

2 Gramática Base y SDT

Las producciones se presentan en BNF compacta. Para cada bloque se indican reglas semánticas cuando aplican y acciones SDT que sintetizan Swift. La idea central es que el árbol no solo se reconoce: también transporta información suficiente para validar y traducir.

2.1 Programa y Declaraciones

programa	-> declaracion programa ε
declaracion	-> importacion decl_variable def_funcion def_clase sentencia
importacion	-> importar ID

Acciones SDT

programa.swift = declaracion.swift + salto de línea + programa1.swift.

programa -> ε sintetiza cadena vacía.

importacion.swift = 'import ' + ID.lexema.

2.2 Variables y Tipos

decl_variable	-> (var sea) tipo ID = expresion
tipo	-> entero real cadena booleano ID
tipo_basico	-> entero real cadena booleano

SEM-1 y SEM-4: declaración de variable

Atributos: ID.nombre, tipo.declarado, expresion.tipo, ambito.actual, tabla_simbolos

Dominio: El identificador no debe existir en el ámbito actual; el tipo de la expresión debe ser compatible con el tipo declarado. entero puede promoverse a real.

Acción: Registrar el símbolo si no hay conflicto; si existe duplicado o incompatibilidad, reportar diagnóstico y bloquear Swift.

Acciones SDT

var ID => var ID: TipoSwift = expr.swift.

sea ID => let ID: TipoSwift = expr.swift.

Mapeo de tipos: entero→Int, real→Double, cadena→String, booleano→Bool.

2.3 Funciones y Parámetros

```
def_funcion    -> funcion tipo_ret ID ( parametros ) hacer bloque fin_funcion
tipo_ret       -> tipo_basico | ε
parametros     -> param_lista | ε
param_lista    -> tipo ID param_resto
param_resto    -> , tipo ID param_resto | ε
sent_retornar  -> retornar expresion
```

SEM-FUNC: ámbito y parámetros

Atributos: funcion.nombre, tipo_ret, parametros.lista, ambito.funcion

Dominio: El nombre de función no debe duplicarse; cada parámetro se declara en el ámbito propio de la función.

Acción: Registrar la función, abrir ámbito local, registrar parámetros, analizar el bloque y cerrar el ámbito.

Acciones SDT

def_funcion.swift = func ID(params.swift) -> TipoRet { bloque.swift }.

Si tipo_ret = ε, se omite la flecha de retorno.

param_lista.swift usa la forma _ ID: TipoSwift para facilitar llamadas estilo Claudio.

sent_retornar.swift = return + expresion.swift.

2.4 Clases, Atributos y Métodos

```
def_clase      -> clase ID herencia_opt hacer cuerpo_clase fin_clase
herencia_opt   -> hereda ID | ε
cuerpo_clase   -> miembro_clase cuerpo_clase | ε
miembro_clase  -> def_atributo | def_metodo
def_atributo   -> atributo tipo ID
def_metodo     -> metodo ID ( parametros ) hacer bloque fin_funcion
```

SEM-CLASE: nombres dentro de clase

Atributos: clase.nombre, atributo.nombre, metodo.nombre, ambito.clase

Dominio: Los nombres declarados dentro del ámbito de clase deben ser únicos.

Acción: Registrar clase, abrir ámbito de clase, registrar miembros y traducir métodos como funciones Swift internas.

Acciones SDT

def_clase.swift = class ID[: Super] { cuerpo.swift }.

def_atributo.swift declara una propiedad Swift con valor por defecto.

default(Int, Double, String, Bool) = 0, 0.0, "", false.

def_metodo.swift = func ID(params.swift) { bloque.swift }.

2.5 Bloques y Sentencias

bloque	-> sentencia bloque'
bloque'	-> sentencia bloque' ε
sentencia	-> sent_id sent_si sent_para sent_mientras sent_retornar sent_imprimir sent_romper sent_continuar decl_variable
sent_id	-> (ID este) resto_id
resto_id	-> = expresion . ID resto_id (argumentos) ε
argumentos	-> arg_lista ε
arg_lista	-> expresion argresto
argresto	-> , expresion argresto ε

SEM-2, SEM-3 y SEM-5: uso y asignación

Atributos: ID.nombre, ID.tipo, ID.inmutable, expresion.tipo, tabla_simbolos

Dominio: Un identificador usado debe existir; una constante declarada con sea no puede reasignarse; el tipo asignado debe ser compatible.

Acción: Resolver el símbolo contra la pila de ámbitos y reportar no declarado, reasignación de constante o incompatibilidad.

Acciones SDT

Asignación: lhs.swift = expresion.swift.

este.campo se traduce a self.campo.

Llamada: ID(args.swift); los argumentos concatenan expresiones con coma.

imprimir(expr) -> print(expr); romper/continuar -> break/continue.

2.6 Condicionales y Ciclos

sent_si	-> si expresion entonces bloque rama_sino fin_si
rama_sino	-> sino bloque ε
sent_para	-> para ID desde expresion hasta expresion paso_opt hacer bloque fin_para
paso_opt	-> paso expresion ε
sent_mientras	-> mientras expresion hacer bloque fin_mientras

SEM-6 y SEM-7: condiciones y límites

Atributos: condicion.tipo, desde.tipo, hasta.tipo, paso.tipo

Dominio: La condición de si/mientras debe ser booleana; desde, hasta y paso de para deben ser numéricos.

Acción: Reportar condición no booleana o límites no numéricos. La variable del ciclo se registra en el ámbito del bloque.

Acciones SDT

si -> if cond.swift { bloque.swift } y sino -> else { bloque.swift }.

para -> for ID in stride(from: desde.swift, through: hasta.swift, by: paso.swift) { bloque.swift }.

paso_opt -> ε sintetiza 1.

mientras -> while cond.swift { bloque.swift }.

2.7 Expresiones

```
expresion      -> expr_or
expr_or        -> expr_and expr_or'
expr_or'       -> o expr_and expr_or' | ε
expr_and       -> expr_rel expr_and'
expr_and'      -> y expr_rel expr_and' | ε
expr_rel       -> expr_add expr_rel'
expr_rel'      -> op_rel expr_add | ε
expr_add       -> expr_mul expr_add'
expr_add'      -> + expr_mul expr_add' | - expr_mul expr_add' | ε
expr_mul       -> expr_pot expr_mul'
expr_mul'      -> * expr_pot expr_mul' | / expr_pot expr_mul' | % expr_pot expr_mul' | ε
expr_pot       -> expr_unaria expr_pot'
expr_pot'      -> ** expr_unaria expr_pot' | ε
expr_unaria    -> no expr_unaria | - expr_unaria | expr_primaria
expr_primaria  -> literal | verdadero | falso | nulo | nuevo ID(argumentos)
               | este sufixo_id | ID sufixo_id | ( expresion )
sufijo_id      -> ( argumentos ) | . ID sufixo_id | ε
```

Acciones SDT

Los operadores aritméticos y relacionales conservan su forma Swift cuando son equivalentes.

y -> &&, o -> ||, no -> !; verdadero/falso/nulo -> true/false/nil.

nuevo ID(args) -> ID(args); este -> self.

a ** b se traduce como pow(Double(a), Double(b)).

3 Reglas Semánticas Implementadas

Regla	Concepto validado	Aplicación en Claudio
SEM-1	Declaración única.	Antes de registrar variables, funciones, clases, atributos o parámetros se revisa el ámbito actual.
SEM-2	Uso de identificadores existentes.	Cada identificador en expresiones, llamadas y asignaciones se resuelve contra la pila de ámbitos.
SEM-3	Inmutabilidad de constantes.	Un símbolo creado con sea queda marcado como inmutable y una asignación posterior se reporta.
SEM-4	Compatibilidad en inicialización.	El tipo declarado se compara con el tipo inferido del valor inicial.
SEM-5	Compatibilidad en asignación.	El tipo almacenado en la tabla se contrasta con la expresión asignada.
SEM-6	Condiciones booleanas.	Las expresiones de si y mientras deben inferirse como booleano.
SEM-7	Límites numéricos en para.	desde, hasta y paso deben inferirse como entero o real.

La tabla de símbolos es el soporte técnico de estas reglas: permite saber dónde fue declarado un nombre, con qué tipo, si es constante y en qué ámbito vive. La inferencia de tipos es conservadora para evitar rechazos incorrectos cuando el compilador no tiene información suficiente.

Relación con la SDT

La traducción no se ejecuta como reemplazo de texto. Primero se valida el AST; después, cada nodo sintetiza un fragmento Swift. Por eso un error SEM bloquea el lenguaje destino aunque la sintaxis sea correcta.

4 Cómo se Materializa en Claudio

La implementación se organiza como una cadena observable de fases. Cada fase agrega evidencia: tokens, árbol/diagnósticos, tabla de símbolos, errores semánticos y salida Swift. La interfaz refleja esta lógica para que el comportamiento del compilador se pueda defender desde el producto desplegado.

Fase	Responsabilidad	Criterio de continuación
Código	Entrada fuente escrita en Claudio.	El texto se envía al análisis elegido.
Léxico	Reconocer tokens y errores de caracteres.	Debe quedar sin errores léxicos para confiar en el parser.
Sintáctico	Validar la forma del programa con la gramática.	Debe producir AST válido para análisis semántico.
Semántico	Validar tipos, ámbitos y reglas SEM.	Debe quedar sin errores para habilitar la SDT.
Swift	Sintetizar el lenguaje destino.	Solo se muestra si las fases previas son válidas.

Compuerta de generación Swift

Situación	Comportamiento esperado
Error léxico	Se reporta el carácter/lexema inválido y se bloquea Swift.
Error sintáctico	Se reporta token encontrado y elemento esperado por la gramática; Swift no se genera.
Error semántico	Se informa la regla SEM involucrada y se bloquea el lenguaje destino.
Sin errores	Se muestra el código Swift y la validación IA opcional.

C Claudio

Lexico

() Desc. Recursivo

Predictivo LL(1)

Semantico

Final. Valido con Swift

Analizar

Ctrl+Enter

✓Codigo

✓Lexico

✓Sintactico

✓Semantico

Swift

Reglas semanticas sobre el AST -- tipos, ambitos, constantes y tabla de simbolos

61 tokens

✓ valida 399 nodos

✓ sem. valida 6 simbolos

1

// Entrega final: debe pasar lexico, sintactico y

2

semantico, y generar Swift

3

funcion entero doble(entero n) hacer

4

retornar n * 2

5

fin_funcion

6

sea entero LIMITE = 4

7

var entero acumulado = 0

8

9

para i desde 1 hasta LIMITE paso 1 hacer

10

var entero valor = doble(i)

11

acumulado = acumulado + valor

12

imprimir(valor)

13

fin_para

14

15

si acumulado > 0 entonces

16

imprimir(acumulado)

17

fin_si

Tokens 61

Tabla Simbolos 6

Swift

Errores

Swift generado

Lenguaje destino listo despues de validar Claudio.

CODIGO → LEXICO → SINTACTICO → SEMANTICO → SWIFT

Detalle SDT

Copiar

Swift

17 LINEAS

14 EFECTIVAS

6 SIMBOLOS

17 REGLAS

CLAUDIO ORIGEN

1

// Entrega final: debe pasar lexico, sintactico

2

funcion entero doble(entero n) hacer

3

retornar n * 2

4

fin_funcion

5

6

sea entero LIMITE = 4

7

var entero acumulado = 0

8

9

para i desde 1 hasta LIMITE paso 1 hacer

10

var entero valor = doble(i)

11

acumulado = acumulado + valor

12

imprimir(valor)

13

fin_para

SWIFT DESTINO

GENERADO

1

// Entrega final: debe pasar lexico, sintactico

2

func doble(_ n: Int) → Int {

3

return n * 2

4

}

5

6

let LIMITE: Int = 4

7

var acumulado: Int = 0

8

9

for i in stride(from: 1, through: LIMITE, by: 1) {

10

var valor: Int = doble(i)

11

acumulado = acumulado + valor

12

print(valor)

13

}

Validacion IA Swift

REVISION DESTINO

lista

La traducción a Swift es sintácticamente razonable y mantiene la intención del programa Claudio: define una función, usa una constante, acumula en un bucle y realiza una condición final. Las estructuras están bien balanceadas y el código parece Swift válido.

Gramatica BNF

Evidencia desde <https://claudio.dautia.com>: caso válido con panel Swift generado, contraste Claudio/Swift y validación IA Swift.

5 Pruebas desde la Interfaz Gráfica

Las pruebas se ejecutan directamente en la interfaz pública: <https://claudio.dautia.com>. Para reproducirlas, se abre la galería de programas, se selecciona el caso indicado y se ejecuta el análisis en modo Semántico.

Caso en la UI	Qué demuestra	Resultado visible
Final. Valido con Swift	Programa correcto con función, constante, ciclo para, acumulador, imprimir y condicional.	La pestaña Swift queda disponible y muestra validación IA.
Final. Error semantico sin Swift	Sintaxis correcta, pero fallas SEM-3, SEM-4 y SEM-6.	La pestaña Errores lista diagnósticos semánticos y Swift queda bloqueado.
Final. Error lexico/sintactico sin Swift	Caracter inválido, condicional incompleto y llamada imprimir mal formada.	La pestaña Swift informa bloqueo por errores previos.

5.1 Caso Válido — Swift Generado

El caso válido demuestra la ruta completa: el compilador acepta léxico, sintaxis y semántica; luego aplica la SDT para producir Swift y habilita la revisión complementaria con OpenAI.

Observación	Interpretación
Fases marcadas como válidas.	La compuerta de generación se abre porque no hay errores acumulados.
Panel Claudio origen / Swift destino.	La traducción conserva la intención del programa fuente.
Validación IA Swift visible.	OpenAI revisa la salida destino después de que Claudio la genera.

5.2 Caso Semántico — Errores y Swift Bloqueado

Este caso tiene forma sintáctica aceptable, pero viola reglas semánticas. La evidencia importante es que Claudio no deja pasar la SDT: muestra diagnósticos SEM y conserva la salida Swift bloqueada.

The screenshot shows the Claudio compiler interface. The top bar includes tabs for 'Codigo', 'Lexico', 'Sintactico', 'Semantico', and 'Swift'. The 'Semantico' tab is active, displaying 'Final. Error semantico sin Swift'. Below the top bar, a status bar shows '22 tokens', 'valida 150 nodos', '3 err sem', and '2 simbolos'. The main editor on the left contains the following code:

```
1 // Error semantico: la sintaxis es correcta, pero la
2 // semantica bloquea Swift
3 sea entero limite = 3
4 limite = 7
5 var entero total = "mucho"
6
7 si total entonces
8   imprimir(total)
9 fin_si
```

The right panel, titled 'Errores 3', lists three semantic errors:

- SEM-3** @ 3:1 limite: No se puede reasignar la constante 'limite' (declarada con 'sea' en fila 2).
Sugerencia determinística: Cambia 'sea' por 'var' en la declaración de 'limite' si necesitas modificarla.
Sugerencia IA (confianza 99%): Estás intentando modificar una constante declarada con 'sea'. En Claudio, una constante no se puede reasignar después de declararla. Corrección: Si el valor debe cambiar, declara 'limite' con 'var' en lugar de 'sea'. Si no debe cambiar, elimina la reasignación.
`var entero limite = 3`
`limite = 7`
- SEM-4** @ 5:12 total: Tipo incompatible en la declaración de 'total': se declaró 'entero' pero el valor asignado es de tipo 'cadena'.
Sugerencia determinística: Cambia el valor a un literal de tipo 'entero' o ajusta el tipo declarado a 'cadena'.
Sugerencia IA (confianza 99%): 'total' se declaró como 'entero', pero se le asignó una cadena. El tipo del valor no coincide con el tipo declarado. Corrección: Cambia el literal por un número entero o declara 'total' como 'cadena' si el texto es el valor correcto.
`var entero total = 10`
`// o`
`var cadena total = "mucho"`
- SEM-6** @ 7:1 si: (partially visible)

A 'Gramatica BNF' button is located at the bottom left of the editor area.

Caso Final. Error semantico sin Swift: Errores muestra SEM-3, SEM-4 y SEM-6 con sugerencias determinísticas e IA.

5.3 Caso Léxico/Sintáctico — Swift Bloqueado

Este caso falla antes de la fase semántica por un carácter no reconocido y por construcciones incompletas. La pestaña Swift no presenta código destino; en su lugar resume los errores que impiden la traducción.

The screenshot shows the Claudio compiler interface. The top bar includes tabs for 'Codigo', 'Lexico', 'Desc. Recursivo', 'Predictivo LL(1)', 'Semantico', and 'Final. Error lexico/sintactico sin Swift'. The 'Semantico' tab is active, and the 'Swift' sub-tab is selected. The code editor on the left contains the following Swift code:

```
1 // Error lexico/sintactico: no debe generarse Swift
2 var entero x = 10
3
4 si x > 5
5     imprimir("Mayor")
6 fin_si
7
8 imprimir(@)
```

The right panel shows the 'Swift' tab with a 'Swift bloqueado' message: 'La salida destino se detiene hasta que las fases previas queden validas.' Below this, a progress bar shows the stages: CODIGO → LEXICO → SINTACTICO → SEMANTICO → SWIFT. The 'SINTACTICO' stage is highlighted, indicating the current phase of the error. Below the progress bar, the errors are listed:

- lexico: 1** sintactico: 3
- lexico** 8:10 @
Caracter no reconocido '@' en fila 8, col 10
- sintactico** 6:5 imprimir
Token inesperado 'imprimir' en condicional si. Esperado: entonces.
- sintactico** 6:1 fin_si
Token inesperado 'fin_si' en imprimir. Esperado:).
- sintactico** 8:11)
Token inesperado ')' en argumento de imprimir. Esperado: NUM_REAL, NUM_ENTERO, (, -, verdadero, no, falso, este, ID, nuevo, CADENA_LIT, nulo.

At the bottom left, there is a button labeled 'Gramatica BNF'.

Caso Final. Error lexico/sintactico sin Swift: el panel Swift queda bloqueado y resume errores léxicos/sintácticos.

6 Bonus IA con OpenAI

La integración con OpenAI funciona como revisión posterior del lenguaje destino. Claudio decide de forma determinística si el programa fuente es válido; solo cuando esa decisión es positiva se revisa el Swift generado.

Aspecto	Descripción
Momento de uso	Después de léxico, sintáctico, semántico y SDT. Si hay errores, no se consulta la revisión de destino.
Información revisada	Código Claudio original y Swift sintetizado por las acciones de traducción.
Criterios	Sintaxis Swift razonable, bloques balanceados, operadores/literales traducidos y conservación de intención.
Resultado en UI	Resumen breve bajo el panel Swift, con estado visible de revisión.
Límite conceptual	La IA no reemplaza la gramática, la tabla de símbolos ni las reglas SEM.

Lectura correcta del bonus

El bonus no convierte a Claudio en un validador probabilístico. La IA se usa para explicar y revisar el Swift ya generado; la aceptación del programa sigue dependiendo de las fases del compilador.

7 Conclusiones

- La entrega final implementa una cadena de compilación completa y observable: código fuente, tokens, análisis sintáctico, tabla de símbolos, errores y Swift.
- La gramática ampliada y las acciones SDT cubren variables, funciones, clases, bloques, condicionales, ciclos y expresiones.
- Las reglas semánticas SEM-1 a SEM-7 se explican desde su concepto y desde su efecto real sobre la generación del lenguaje destino.
- La interfaz pública permite reproducir los casos de prueba desde la galería visual de Claudio, sin depender de pasos técnicos externos.
- OpenAI aporta una revisión complementaria del Swift, pero la validez del programa depende de las reglas determinísticas del compilador.